Chuleta de examen – Juego de sumas (Unidad 3)

Cambios típicos que pueden pedir sobre `juego.js`, `index.html` y `estilos.css`. Cada bloque dice **dónde** tocar y **qué** escribir, usando los nombres reales del proyecto.

> ⚠ **TRAMPA IMPORTANTE – `cargarPartida`**: valida la partida guardada con `Number.isInteger(partida.puntos)`, `partida.puntos === partida.historial.length` y `numero1`/`numero2` entre 1 y 10. Si cambias **puntos decimales**, **restar puntos**, **puntos por acierto** o **rango de números**, quita/adapta esas líneas o al recargar la partida no se recuperará (vuelve el formulario). Lo mismo en `cargarRecord`: `Number.isInteger(guardado)` rechaza récords decimales.

0. Mapa rápido del código

Función	Qué hace
`validarPerfil(evento)`	submit del formulario: trim, fecha válida, edad ≥ `EDAD_MINIMA`
`calcularEdad(nacimiento, hoy)`	resta años y descuenta 1 si no ha llegado el cumpleaños
`mostrarPanel()` / `cambiarColorPanel()`	nombre en mayúsculas, edad, color azul (≥18) / verde
`siguienteMision()`	`Math.floor(Math.random() * 10) + 1` para dos números
`comprobarRespuesta(evento)`	compara, `push` al historial, +1 punto, récord, guarda
`abrirAyuda()`	`window.open` + `document.write` + `textContent`
`guardarPartida()` / `cargarPartida()`	`sessionStorage` + `JSON.stringify` / `JSON.parse`
`reiniciarPartida()`	`sessionStorage.removeItem` + `location.reload()`
`cargarRecord()` / `actualizarRecord()`	`localStorage` + `Math.max`
`aplicarLetra()` / `cargarPreferencia()` / `guardarPreferencia()`	cookie `letraSumas` 30 días

Claves de almacenamiento: `partidaSumas` (session), `recordSumas` (local), cookie `letraSumas`.

1. Redondear la puntuación (Math)

Método	2.5	2.4	-2.5	Uso
`Math.round(x)`	3	2	-2	al más cercano
`Math.floor(x)`	2	2	-3	hacia abajo
`Math.ceil(x)`	3	3	-2	hacia arriba
`Math.trunc(x)`	2	2	-2	quita decimales
`x.toFixed(2)`	"2.50"	"2.40"	"-2.50"	**devuelve String**, para mostrar

Redondear a 2 decimales como número: `Math.round(x * 100) / 100`.

Ejemplo: puntos con bonus aleatorio y redondeo (en `comprobarRespuesta`, sustituye la línea `puntuacion = puntuacion + 1;`):

```
        const bonus = Math.random();                  // 0 … 0.999
        puntuacion = Math.round((puntuacion + 1 + bonus) * 10) / 10; // 1 decimal
        textoPuntuacion.textContent = "Puntos: " + puntuacion.toFixed(1);
        mensajeJuego.textContent = "¡Correcto! +" + (1 + bonus).toFixed(1) + " puntos.";
```

Adaptar: en `cargarPartida` cambiar `!Number.isInteger(partida.puntos)` por `typeof partida.puntos !== "number"` y **borrar** `partida.puntos !== partida.historial.length`. En `cargarRecord` cambiar `Number.isInteger(guardado)` por `!isNaN(guardado)`.

Puntos según dificultad (suma grande da más): `puntuacion += Math.ceil((numero1 + numero2) / 5);`

Restar al fallar (en el `else` de `comprobarRespuesta`):

```
        puntuacion = Math.max(0, puntuacion - 1);    // nunca negativo
        textoPuntuacion.textContent = "Puntos: " + puntuacion;
        guardarPartida();
```

(Quitar `partida.puntos !== partida.historial.length` en `cargarPartida`.)

2. Fechas (Date)

Métodos: `getFullYear()`, `getMonth()` (**0 = enero**, sumar 1 para mostrar), `getDate()` (día del mes), `getDay()` (día de la semana, **0 = domingo**), `getHours()`, `getMinutes()`, `getSeconds()`, `getTime()` (ms desde 1970), `setHours(0,0,0,0)`, `toLocaleDateString()`, `toLocaleTimeString()`.

Mostrar la fecha actual – en `index.html` debajo del `<h1>`: `<p id="fecha-actual"></p>` y en `juego.js` (al final):

```
const textoFecha = document.getElementById("fecha-actual");
function mostrarFechaActual() {
    const ahora = new Date();
    textoFecha.textContent = "Hoy es " + ahora.toLocaleDateString("es-ES", {
        weekday: "long", day: "numeric", month: "long", year: "numeric"
    });
}
mostrarFechaActual();
```

Formato manual `dd/mm/aaaa`:

```
const d = new Date();
const dia = String(d.getDate()).padStart(2, "0");
const mes = String(d.getMonth() + 1).padStart(2, "0");   // +1 !!
textoFecha.textContent = dia + "/" + mes + "/" + d.getFullYear();
```

Reloj que se actualiza cada segundo:

```
setInterval(function () {
    textoFecha.textContent = new Date().toLocaleTimeString("es-ES");
}, 1000);
```

Fecha/hora en cada acierto del historial (en `comprobarRespuesta`):

```
const hora = new Date().toLocaleTimeString("es-ES");
historial.push(numero1 + " + " + numero2 + " = " + respuesta + " (" + hora + ")");
```

Días que faltan para su cumpleaños (guardar `nacimiento` en `validarPerfil`):

```
function diasHastaCumple(nacimiento) {
    const hoy = new Date(); hoy.setHours(0, 0, 0, 0);
    let cumple = new Date(hoy.getFullYear(), nacimiento.getMonth(), nacimiento.getDate());
    if (cumple < hoy) cumple.setFullYear(hoy.getFullYear() + 1);
    return Math.round((cumple - hoy) / (1000 * 60 * 60 * 24));
}
```

Cambiar edad mínima: `const EDAD_MINIMA = 16;` + texto de `index.html` ("al menos 16 años"). **Edad máxima**: en `validarPerfil` tras calcular la edad: `if (edad < EDAD_MINIMA || edad > 99) return;`

---

3. Modificar el color de fondo (style / classList)

Toda la página: `document.body.style.backgroundColor = "#fde68a";` Solo la caja blanca: `document.querySelector("main").style.backgroundColor = "lightyellow";`

Más tramos de edad en `cambiarColorPanel`:

```
if (edadJugador >= 65) {
    panelJugador.style.backgroundColor = "#fef3c7";   // amarillo
} else if (edadJugador >= 18) {
    panelJugador.style.backgroundColor = "#dbeafe";   // azul
} else {
    panelJugador.style.backgroundColor = "#dcfce7";   // verde
}
```

Fondo según acierto/fallo (en `comprobarRespuesta`):

```
document.body.style.backgroundColor = "#bbf7d0";   // en el if (acierto)
document.body.style.backgroundColor = "#fecaca";   // en el else (fallo)
```

Color aleatorio:

```
function colorAleatorio() {
    const r = Math.floor(Math.random() * 256);
    const g = Math.floor(Math.random() * 256);
    const b = Math.floor(Math.random() * 256);
    document.body.style.backgroundColor = "rgb(" + r + ", " + g + ", " + b + ")";
}
```

Selector de color del usuario – HTML: `<input id="color-fondo" type="color" value="#eef2f6">`

```
const selectorColor = document.getElementById("color-fondo");
selectorColor.addEventListener("input", function () {
    document.body.style.backgroundColor = selectorColor.value;
});
```

Con clases CSS – en `estilos.css`: `.oscuro { background-color: #111827; color: white; }`

```
document.body.classList.toggle("oscuro");   // add / remove / toggle / contains
```

Guardar el color elegido (persistente): `localStorage.setItem("colorFondo", selectorColor.value);` y al cargar: `const c = localStorage.getItem("colorFondo"); if (c) document.body.style.backgroundColor = c;`

4. Mostrar info por consola (F12 → Console)

```
console.log("Jugador:", nombreJugador, "Edad:", edadJugador);
console.info("Partida recuperada");
console.warn("Respuesta vacía");
console.error("No se pudo guardar", error);       // dentro de los catch
console.table(historial);                          // arrays/objetos en tabla
console.table({ nombre: nombreJugador, puntos: puntuacion, record: record });
console.group("Misión"); console.log(numero1, numero2); console.groupEnd();
console.count("aciertos");                         // cuenta las veces que se llama
console.time("respuesta"); /* … */ console.timeEnd("respuesta");
console.clear();
```

Sitios típicos: final de `validarPerfil` (perfil válido), `siguienteMision` (números generados), `comprobarRespuesta` (respuesta y si es correcta), `guardarPartida` (`console.log(partida)`), los `catch (error)` → `console.error(error)`. Plantilla literal: `console.log(`Suma: ${numero1} + ${numero2} = ${numero1 + numero2}`);`

5. Obtener y modificar elementos del DOM

Obtener:

```
document.getElementById("mensaje");
document.querySelector("#panel-jugador h2");      // el primero que coincide (selector CSS)
document.querySelectorAll("button");              // NodeList → forEach
document.getElementsByTagName("p");
document.getElementsByClassName("clase");
```

Modificar:

```
elemento.textContent = "texto";          // seguro (no interpreta HTML)
elemento.innerHTML = "<strong>hola</strong>";  // interpreta HTML (no con datos del usuario)
elemento.style.color = "red";            // propiedades CSS en camelCase
elemento.classList.add("activo");
elemento.setAttribute("title", "Ayuda"); elemento.getAttribute("id");
elemento.hidden = true;                  // ocultar / mostrar
campoRespuesta.value = "";               // inputs
botonComprobar.disabled = true;
```

Crear / eliminar:

```
const nuevo = document.createElement("p");
nuevo.textContent = "Nuevo párrafo";
document.querySelector("main").appendChild(nuevo);   // o append / prepend
nuevo.remove();
```

Historial como lista `<ul>` en vez de `join` – HTML: cambiar `<p id="historial">` por `<ul id="historial"></ul>`

```
function pintarHistorial() {
    textoHistorial.innerHTML = "";                // vaciar
    historial.forEach(function (suma) {
        const li = document.createElement("li");
        li.textContent = suma;
        textoHistorial.appendChild(li);
    });
}
```

y llamar `pintarHistorial()` donde ahora está `textoHistorial.textContent = historial.join(" | ");`.

Recorrer varios elementos:

```
document.querySelectorAll("#zona-juego button").forEach(function (b) {
    b.style.backgroundColor = "#16a34a";
});
```

Cambiar el título: `document.querySelector("h1").textContent = "Juego de " + nombreJugador;` **Título de la pestaña:** `document.title = "Puntos: " + puntuacion;`

Botón creado desde JS con evento:

```
const boton = document.createElement("button");
boton.type = "button";
boton.textContent = "Borrar récord";
boton.addEventListener("click", function () {
    localStorage.removeItem("recordSumas");
    record = 0;
    textoRecord.textContent = "Récord: 0";
});
zonaJuego.appendChild(boton);
```

Eventos útiles: `click`, `submit` (+ `evento.preventDefault()`), `change`, `input`, `keydown`, `load`, `message`, `storage`.

6. Cambiar entre localStorage y sessionStorage

	`sessionStorage`	`localStorage`
Dura	hasta cerrar la pestaña	permanente (hasta borrarlo)
Ámbito	solo esa pestaña	todas las pestañas del mismo origen
Ahora guarda	la partida (`partidaSumas`)	el récord (`recordSumas`)

API idéntica: `setItem(clave, texto)`, `getItem(clave)`, `removeItem(clave)`, `clear()`, `length`, `key(i)`. **Solo guarda texto** → objetos con `JSON.stringify` / `JSON.parse`; números con `String` / `Number`.

Partida permanente (que siga al cerrar el navegador): cambiar `sessionStorage` por `localStorage` en `guardarPartida`, `cargarPartida` y `reiniciarPartida` (3 sitios). Texto del mensaje: "Partida guardada en este navegador."

Récord solo de la sesión: cambiar `localStorage` por `sessionStorage` en `cargarRecord` y `actualizarRecord` (2 sitios).

Truco limpio: una variable para elegir el almacén:

```
const almacen = localStorage;   // o sessionStorage
almacen.setItem("partidaSumas", JSON.stringify(partida));
```

Borrar todo: `localStorage.clear(); sessionStorage.clear();` **Ver en el navegador:** F12 → Application (Aplicación) → Storage → Local/Session Storage / Cookies.

Cookies (recordatorio):

```
document.cookie = "clave=valor; max-age=2592000; path=/";   // crear (30 días)
document.cookie = "clave=; max-age=0; path=/";              // borrar
document.cookie.split(";")                                    // leer → array "clave=valor"
```

7. Mensajes entre ventanas (postMessage)

`otraVentana.postMessage(dato, origenDestino)` envía; quien recibe escucha `window.addEventListener("message", …)`.

- Desde la principal a la ayuda: `ventana.postMessage(...)` (referencia que devuelve `window.open`).
- Desde la ayuda a la principal: `window.opener.postMessage(...)`.
- Comprobar siempre `evento.origin === window.location.origin` y leer `evento.data`.

Versión sin archivos nuevos (todo en `juego.js`). Sustituir `abrirAyuda` por:

```
        let ventanaAyuda = null;   // guardamos la referencia para enviar mensajes después

        function abrirAyuda() {
            ventanaAyuda = window.open("", "_blank", "width=450,height=400");
            if (!ventanaAyuda) {
                mensajeJuego.textContent = "Permite las ventanas emergentes para ver la ayuda.";
                return;
            }
            ventanaAyuda.document.write('<!DOCTYPE html><html lang="es"><head><meta charset="UTF-8"><title>Ayuda</title></head><
<h1>Cómo jugar</h1><p id="datos"></p><button id="saludar">Enviar mensaje al juego</button></body></html>');
            ventanaAyuda.document.close();

            // La ayuda ESCUCHA mensajes de la principal
            ventanaAyuda.addEventListener("message", function (evento) {
                if (evento.origin !== window.location.origin) return;
                ventanaAyuda.document.getElementById("datos").textContent =
                    "Jugador: " + evento.data.nombre + ". Puntos: " + evento.data.puntos;
            });

            // La ayuda ENVÍA un mensaje a la principal (opener)
            ventanaAyuda.document.getElementById("saludar").addEventListener("click", function () {
                ventanaAyuda.opener.postMessage("¡Hola desde la ayuda!", window.location.origin);
            });

            enviarDatosAyuda();
        }

        function enviarDatosAyuda() {
            if (ventanaAyuda && !ventanaAyuda.closed) {
                ventanaAyuda.postMessage({ nombre: nombreJugador, puntos: puntuacion },
                                         window.location.origin);
            }
        }

        // La principal ESCUCHA mensajes de la ayuda
        window.addEventListener("message", function (evento) {
            if (evento.origin !== window.location.origin) return;
            if (typeof evento.data === "string") {
                mensajeJuego.textContent = "Mensaje recibido: " + evento.data;
            }
        });
```

Y en `comprobarRespuesta`, tras sumar el punto, llamar `enviarDatosAyuda();` → la ayuda se actualiza en directo.

Versión con archivo `ayuda.html` propio (si lo piden):

```
        // juego.js
        ventanaAyuda = window.open("ayuda.html", "ayuda", "width=450,height=400");
        window.addEventListener("message", function (e) {
            if (e.origin !== window.location.origin) return;
            if (e.data === "lista") enviarDatosAyuda();   // la ayuda avisa cuando ha cargado
        });
```

```
        <!-- ayuda.html -->
        <p id="datos"></p>
        <script>
          window.addEventListener("message", function (e) {
              if (e.origin !== window.location.origin) return;
              document.getElementById("datos").textContent =
                  "Jugador: " + e.data.nombre + ". Puntos: " + e.data.puntos;
          });
          window.opener.postMessage("lista", window.location.origin);
        </script>
```

(Hace falta el "lista" porque si se envía antes de que `ayuda.html` cargue, el mensaje se pierde.)

Otras formas de comunicar ventanas/pestañas:

- Acceso directo (mismo origen): `ventana.document.getElementById("datos").textContent = …` (lo que ya hace el proyecto).
- `ventana.close()` cierra la ayuda; `ventana.closed` dice si está cerrada.
- Entre **pestañas**: evento `storage` (salta en las OTRAS pestañas cuando cambia localStorage):

```
        window.addEventListener("storage", function (e) {
            if (e.key === "recordSumas") textoRecord.textContent = "Récord: " + e.newValue;
        });
```

- `BroadcastChannel`:

```
        const canal = new BroadcastChannel("sumas");
        canal.postMessage({ puntos: puntuacion });
        canal.onmessage = function (e) { console.log("Otra pestaña:", e.data); };
```

8. Otros cambios probables

Cambiar la operación a resta / multiplicación – en `siguienteMision` cambiar `" + "` por `" × "` y en `comprobarRespuesta` `numero1 + numero2` por `numero1 * numero2` (y el texto del historial). Operación aleatoria:

```
const operaciones = ["+", "-", "*"];
operacion = operaciones[Math.floor(Math.random() * operaciones.length)];  // let operacion global
function resultado() {
    if (operacion === "+") return numero1 + numero2;
    if (operacion === "-") return numero1 - numero2;
    return numero1 * numero2;
}
```

Rango de números (ej. 1–20): `Math.floor(Math.random() * 20) + 1`. Fórmula general entre `min` y `max`: `Math.floor(Math.random() * (max - min + 1)) + min`. Adaptar `partida.numero1 > 10` → `> 20` en `cargarPartida`.

Límite de intentos:

```
let intentos = 3;                        // global; en siguienteMision → intentos = 3;
// en el else de comprobarRespuesta:
intentos = intentos - 1;
if (intentos === 0) {
    mensajeJuego.textContent = "Sin intentos. Era " + (numero1 + numero2);
    botonComprobar.disabled = true;
} else {
    mensajeJuego.textContent = "Fallo. Te quedan " + intentos + " intentos.";
}
```

Temporizador por suma (`setTimeout` / `clearTimeout`):

```
let temporizador;
// en siguienteMision:
clearTimeout(temporizador);
temporizador = setTimeout(function () {
    mensajeJuego.textContent = "¡Tiempo agotado!";
    botonComprobar.disabled = true;
}, 10000);
// en el acierto: clearTimeout(temporizador);
```

Ganar al llegar a X puntos: `if (puntuacion >= 10) { mensajeJuego.textContent = "¡Has ganado!"; botonComprobar.disabled = true; botonSiguiente.disabled = true; }`

Confirmar antes de reiniciar: al inicio de `reiniciarPartida`: `if (!confirm("¿Seguro?")) return;`

Aviso de nuevo récord (en `actualizarRecord`, antes de `record = Math.max(...)`): `if (puntuacion > record) mensajeRecord.textContent = "¡Nuevo récord!";` (y quitar la línea que lo vacía).

Nombre con primera letra mayúscula (en `mostrarPanel`): `nombreJugador.charAt(0).toUpperCase() + nombreJugador.slice(1).toLowerCase()` Otros de String: `length`, `toLowerCase`, `includes`, `indexOf`, `slice`, `substring`, `replace`, `split`, `padStart`, `repeat`.

Nombre mínimo 3 letras: en `validarPerfil`: `if (nombre.length < 3) return;`

Cookie 7 días en vez de 30: `max-age=604800` (7 × 24 × 60 × 60) y cambiar el texto del mensaje. **Tres tamaños de letra:** añadir `<option value="enorme">` y un `else if (selectorLetra.value === "enorme") document.body.style.fontSize = "24px";` (en `cargarPreferencia` comparar también con `"letraSumas=enorme"`).

Media de puntos / estadísticas (Number, Math): `Math.max(...array)`, `Math.min(...array)`, `Math.abs`, `Math.pow(a, b)`, `Math.sqrt`, `parseInt("12px")`, `parseFloat`, `Number.isNaN`, `isNaN`.

9. Recordatorios rápidos

- `"use strict"` arriba: obliga a declarar variables.
- `defer` en el `<script>` del `head` → el DOM ya existe cuando se ejecuta el JS.
- `const` no se reasigna; `let` sí. `===` compara valor y tipo.
- `evento.preventDefault()` evita que el `submit` recargue la página.
- `return` corta la función (validaciones).
- `input.value` siempre es **String** → `Number(...)`.
- `try { … } catch (error) { … }` → seguir funcionando si falla el almacenamiento.
- Probar con **Live Server** (cookies y ventanas no van bien con `file://`).
- Tras cada cambio: F12 → Console para ver errores; Application para ver storage/cookies.

Anexo – Código original completo

index.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <!-- Adapta el ancho de la página a la pantalla del móvil o del ordenador. -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Juego de misiones - Unidad 3</title>
    <link rel="stylesheet" href="estilos.css">
    <!-- defer espera a que el HTML esté preparado antes de ejecutar JavaScript.
         Así podemos acceder al formulario y al párrafo aunque el script esté en head. -->
    <script src="juego.js" defer></script>
</head>
<body>
    <main>
        <h1>Juego de misiones</h1>
        <p>Resuelve sumas sencillas y consigue puntos.</p>

        <label for="tamano-letra">Tamaño de letra</label>
        <select id="tamano-letra">
            <option value="normal">Normal</option>
            <option value="grande">Grande</option>
        </select>
        <p id="mensaje-preferencia" aria-live="polite"></p>

        <!-- Cada label se une a su campo mediante for e id.
             required indica que el campo no puede quedar vacío. -->
        <!-- novalidate deja la validación a JavaScript para mostrar nuestro mensaje. -->
        <form id="formulario-jugador" novalidate>
            <label for="nombre">Nombre del jugador</label>
            <input id="nombre" type="text" required>

            <label for="fecha-nacimiento">Fecha de nacimiento</label>
            <input id="fecha-nacimiento" type="date" required>

            <p>Debes tener al menos 12 años para jugar.</p>
            <button id="boton-inicio" type="submit">Comenzar</button>
        </form>

        <!-- JavaScript cambiará este mensaje al pulsar el botón.
             aria-live permite que los lectores de pantalla anuncien el cambio. -->
        <p id="mensaje" aria-live="polite">Introduce tus datos y pulsa Comenzar.</p>

        <!-- hidden oculta el panel hasta que el jugador introduce datos válidos. -->
        <section id="panel-jugador" hidden>
            <h2>Tu perfil</h2>
            <p id="nombre-jugador"></p>
            <p id="edad-jugador"></p>
        </section>
        <!-- El juego solo se muestra después de validar el perfil. -->
        <section id="zona-juego" hidden>
            <h2>Resuelve la suma</h2>
            <p id="puntuacion" aria-live="polite">Puntos: 0</p>
            <p id="record" aria-live="polite">Récord: 0</p>
            <p id="mensaje-record" aria-live="polite"></p>
            <p id="suma"></p>
            <form id="formulario-respuesta" novalidate>
                <label for="respuesta">Tu respuesta</label>
                <input id="respuesta" type="number" step="1" required>
                <button id="boton-comprobar" type="submit">Comprobar</button>
                <button id="boton-siguiente" type="button">Siguiente</button>
            </form>
            <p id="mensaje-juego" aria-live="polite"></p>
            <button id="boton-ayuda" type="button">Ayuda</button>
            <button id="boton-reiniciar" type="button">Nueva partida</button>
            <p id="mensaje-guardado" aria-live="polite"></p>
            <h3>Sumas resueltas</h3>
            <p id="historial">Todavía no has resuelto ninguna suma.</p>
        </section>
    </main>
</body>
</html>
```

juego.js

```javascript
"use strict";

// defer hace que este código se ejecute después de crear los elementos del HTML.
```

```
const formularioJugador = document.getElementById("formulario-jugador");
const campoNombre = document.getElementById("nombre");
const campoFecha = document.getElementById("fecha-nacimiento");
const mensaje = document.getElementById("mensaje");
const panelJugador = document.getElementById("panel-jugador");
const textoNombre = document.getElementById("nombre-jugador");
const textoEdad = document.getElementById("edad-jugador");
const zonaJuego = document.getElementById("zona-juego");
const textoSuma = document.getElementById("suma");
const formularioRespuesta = document.getElementById("formulario-respuesta");
const campoRespuesta = document.getElementById("respuesta");
const botonComprobar = document.getElementById("boton-comprobar");
const botonSiguiente = document.getElementById("boton-siguiente");
const mensajeJuego = document.getElementById("mensaje-juego");
const textoHistorial = document.getElementById("historial");
const textoPuntuacion = document.getElementById("puntuacion");
const botonAyuda = document.getElementById("boton-ayuda");
const botonReiniciar = document.getElementById("boton-reiniciar");
const mensajeGuardado = document.getElementById("mensaje-guardado");
const textoRecord = document.getElementById("record");
const mensajeRecord = document.getElementById("mensaje-record");
const EDAD_MINIMA = 12;

// Guardaremos estos datos solo cuando el perfil sea válido.
// let permite cambiar su valor; const se utiliza para valores que no reasignamos.
let nombreJugador = "";
let edadJugador = 0;
let numero1 = 0;
let numero2 = 0;
let resuelta = false;
let puntuacion = 0;
let record = 0;

// Un array guarda varios datos. Aquí añadiremos las sumas acertadas.
let historial = [];

function calcularEdad(nacimiento, hoy) {
    // Primero restamos los años. Luego comprobamos si ya ha cumplido años.
    let edad = hoy.getFullYear() - nacimiento.getFullYear();
    const mesActual = hoy.getMonth();
    const mesNacimiento = nacimiento.getMonth();

    // Si su cumpleaños todavía no ha llegado, restamos un año.
    if (mesActual < mesNacimiento ||
        (mesActual === mesNacimiento && hoy.getDate() < nacimiento.getDate())) {
        edad = edad - 1;
    }

    return edad;
}

function cambiarColorPanel() {
    // style.backgroundColor cambia el fondo del panel desde JavaScript.
    // Usamos un if/else sencillo: azul para mayores de edad y verde para menores.
    if (edadJugador >= 18) {
        panelJugador.style.backgroundColor = "#dbeafe"; // Azul claro.
    } else {
        panelJugador.style.backgroundColor = "#dcfce7"; // Verde claro.
    }
}

function mostrarPanel() {
    // toUpperCase es un método de String: convierte el nombre a mayúsculas.
    // textContent muestra texto, sin interpretar lo escrito como código HTML.
    textoNombre.textContent = "Jugador: " + nombreJugador.toUpperCase();

    // La edad ya se calculó con los métodos de Date en calcularEdad.
    textoEdad.textContent = "Edad: " + edadJugador + " años";
    cambiarColorPanel();
    panelJugador.hidden = false; // Hace visible el panel.
}

function validarPerfil(evento) {
    // Evita que el formulario recargue la página al pulsar Comenzar o Enter.
    evento.preventDefault();

    // trim elimina los espacios de los extremos: "   " se convierte en "".
    const nombre = campoNombre.value.trim();
    const fecha = campoFecha.value;

    // Empezamos denegando el acceso. Solo lo permitimos si pasan todas las pruebas.
    mensaje.textContent = "Acceso denegado";

    if (nombre === "" || fecha === "" || !campoFecha.validity.valid) {
        return; // Termina la función: no seguimos con un dato vacío o incorrecto.
```

```
    }

    // El campo date devuelve año-mes-día. Añadimos la hora para usar la fecha local.
    const nacimiento = new Date(fecha + "T00:00:00");
    const hoy = new Date();
    hoy.setHours(0, 0, 0, 0); // Quitamos la hora para comparar solo los días.

    // isNaN detecta una fecha que JavaScript no ha podido interpretar.
    // La comparación del día también rechaza fechas como el 30 de febrero,
    // que JavaScript puede convertir automáticamente en un día de marzo.
    const diaEscrito = Number(fecha.slice(8, 10));
    if (isNaN(nacimiento.getTime()) || nacimiento.getDate() !== diaEscrito ||
        nacimiento >= hoy) {
        return; // Rechazamos fechas imposibles, la fecha de hoy y fechas futuras.
    }

    const edad = calcularEdad(nacimiento, hoy);
    if (edad < EDAD_MINIMA) {
        return; // Debe haber cumplido al menos 12 años.
    }

    // Solo llegamos aquí si todos los datos son correctos.
    nombreJugador = nombre;
    edadJugador = edad;
    formularioJugador.hidden = true;
    mensaje.textContent = "Acceso permitido. Perfil creado.";
    mostrarPanel();
    zonaJuego.hidden = false;
    siguienteMision();
}

function siguienteMision() {
    if (nombreJugador === "") {
        return; // No generamos una misión sin un perfil válido.
    }

    // random da un decimal desde 0 hasta menos de 1.
    // Multiplicamos por 10, floor quita los decimales y +1 da un número del 1 al 10.
    numero1 = Math.floor(Math.random() * 10) + 1;
    numero2 = Math.floor(Math.random() * 10) + 1;
    textoSuma.textContent = numero1 + " + " + numero2 + " = ?";
    campoRespuesta.value = "";
    resuelta = false; // Esta nueva suma todavía no se ha acertado.
    botonComprobar.disabled = false;
    mensajeJuego.textContent = "";
    campoRespuesta.focus();
    guardarPartida();
}

function comprobarRespuesta(evento) {
    evento.preventDefault();

    if (nombreJugador === "" || resuelta) {
        return; // Una suma ya acertada no se puede registrar dos veces.
    }

    // value es texto. Number lo convierte a un número para comparar la respuesta.
    const respuesta = Number(campoRespuesta.value);
    if (campoRespuesta.value.trim() === "" || !Number.isInteger(respuesta)) {
        mensajeJuego.textContent = "Escribe un número entero.";
        return;
    }

    if (respuesta === numero1 + numero2) {
        resuelta = true;
        botonComprobar.disabled = true;
        // push añade un dato al final del array; join une los datos para mostrarlos.
        historial.push(numero1 + " + " + numero2 + " = " + respuesta);
        textoHistorial.textContent = historial.join(" | ");
        // Solo sumamos aquí: el control de resuelta impide repetir el punto.
        puntuacion = puntuacion + 1;
        textoPuntuacion.textContent = "Puntos: " + puntuacion;
        mensajeJuego.textContent = "¡Correcto! Has ganado 1 punto. Pulsa Siguiente.";
        actualizarRecord();
        guardarPartida();
    } else {
        mensajeJuego.textContent = "No es correcto. Inténtalo otra vez.";
    }
}

// La función se ejecuta al enviar el formulario, no al cargar la página.
formularioJugador.addEventListener("submit", validarPerfil);

formularioRespuesta.addEventListener("submit", comprobarRespuesta);
botonSiguiente.addEventListener("click", siguienteMision);
```

```
        function abrirAyuda() {
            // Abrimos la ventana directamente desde el clic para evitar bloqueos.
            const ventana = window.open("", "_blank", "width=450,height=400");
            if (!ventana) {
                mensajeJuego.textContent = "Permite las ventanas emergentes para ver la ayuda.";
                return;
            }

            // El HTML es fijo. El nombre se escribe después con textContent, de forma segura.
            ventana.document.write('<!DOCTYPE html><html lang="es"><head><meta charset="UTF-8"><title>Ayuda</title></head><body>
<h1>Cómo jugar</h1><p id="datos"></p><p>Suma los dos números y pulsa Comprobar. Cada acierto da 1 punto.</p><p>Si fallas,
puedes volver a intentarlo. Siguiente muestra otra suma.</p><p>Cada suma solo puede dar un punto. Puedes cerrar esta ventana
para seguir jugando.</p></body></html>');
            ventana.document.close();
            ventana.document.getElementById("datos").textContent =
                "Jugador: " + nombreJugador + ". Puntos actuales: " + puntuacion + ".";
        }

        botonAyuda.addEventListener("click", abrirAyuda);

        function guardarPartida() {
            // Un objeto agrupa los datos de la partida con nombres fáciles de reconocer.
            const partida = {
                nombre: nombreJugador, edad: edadJugador, puntos: puntuacion,
                numero1: numero1, numero2: numero2, resuelta: resuelta, historial: historial
            };
            try {
                // sessionStorage guarda texto en esta pestaña. JSON.stringify convierte el objeto a texto.
                sessionStorage.setItem("partidaSumas", JSON.stringify(partida));
                mensajeGuardado.textContent = "Partida guardada en esta pestaña.";
            } catch (error) {
                // Si el navegador bloquea el almacenamiento, se puede seguir jugando.
                mensajeGuardado.textContent = "No se ha podido guardar la partida.";
            }
        }

        function cargarPartida() {
            try {
                const guardada = sessionStorage.getItem("partidaSumas");
                if (!guardada) {
                    return; // Si no hay partida, dejamos visible el formulario inicial.
                }
                // JSON.parse convierte el texto guardado otra vez en un objeto.
                const partida = JSON.parse(guardada);
                // Comprobaciones básicas para no cargar datos incompletos o dañados.
                if (!partida || typeof partida.nombre !== "string" || partida.nombre.trim() === "" ||
                    !Number.isInteger(partida.edad) || partida.edad < EDAD_MINIMA ||
                    !Number.isInteger(partida.puntos) || partida.puntos < 0 ||
                    !Number.isInteger(partida.numero1) || partida.numero1 < 1 || partida.numero1 > 10 ||
                    !Number.isInteger(partida.numero2) || partida.numero2 < 1 || partida.numero2 > 10 ||
                    typeof partida.resuelta !== "boolean" || !Array.isArray(partida.historial) ||
                    partida.puntos !== partida.historial.length) {
                    return;
                }

                nombreJugador = partida.nombre;
                edadJugador = partida.edad;
                puntuacion = partida.puntos;
                numero1 = partida.numero1;
                numero2 = partida.numero2;
                resuelta = partida.resuelta;
                historial = partida.historial;
                formularioJugador.hidden = true;
                mostrarPanel();
                zonaJuego.hidden = false;
                textoSuma.textContent = numero1 + " + " + numero2 + " = ?";
                textoPuntuacion.textContent = "Puntos: " + puntuacion;
                if (historial.length > 0) {
                    textoHistorial.textContent = historial.join(" | ");
                }
                // Restaurar resuelta impide ganar otra vez el punto después de recargar.
                botonComprobar.disabled = resuelta;
                mensaje.textContent = "Partida recuperada.";
                if (resuelta) {
                    mensajeJuego.textContent = "Esta suma ya está resuelta. Pulsa Siguiente.";
                }
            } catch (error) {
                mensaje.textContent = "No se pudo recuperar la partida. Introduce tus datos.";
            }
        }

        function reiniciarPartida() {
            try {
                // Solo borramos nuestra partida, no otros datos de la pestaña.
```

```
            sessionStorage.removeItem("partidaSumas");
            window.location.reload();
        } catch (error) {
            mensajeGuardado.textContent = "No se ha podido borrar la partida guardada.";
        }
    }

    botonReiniciar.addEventListener("click", reiniciarPartida);
    // Se ejecuta al cargar la página: recupera la misma suma sin generar otra.
    cargarPartida();

    // localStorage conserva el récord aunque cierres el navegador.
    // Number convierte el texto guardado a número; rechazamos valores dañados.
    function cargarRecord() {
        try {
            const guardado = Number(localStorage.getItem("recordSumas"));
            if (Number.isInteger(guardado) && guardado >= 0) {
                record = Math.max(record, guardado);
            }
        } catch (error) {
            mensajeRecord.textContent = "No se pudo leer el récord guardado.";
        }
        textoRecord.textContent = "Récord: " + record;
    }

    function actualizarRecord() {
        // Otra pestaña puede haber mejorado el récord: lo leemos antes de guardar.
        cargarRecord();
        // Math.max elige el mayor: una partida con menos puntos no reduce el récord.
        record = Math.max(record, puntuacion);
        textoRecord.textContent = "Récord: " + record;
        try {
            localStorage.setItem("recordSumas", String(record));
            mensajeRecord.textContent = "";
        } catch (error) {
            mensajeRecord.textContent = "El récord solo se conservará mientras esta página siga abierta.";
        }
    }

    // Se carga después de recuperar la partida para incluir sus puntos en el récord.
    cargarRecord();
    actualizarRecord();

    // La cookie guarda solo una preferencia, nunca el nombre ni la fecha de nacimiento.
    const selectorLetra = document.getElementById("tamano-letra");
    const mensajePreferencia = document.getElementById("mensaje-preferencia");

    function aplicarLetra() {
        if (selectorLetra.value === "grande") {
            document.body.style.fontSize = "20px";
        } else {
            document.body.style.fontSize = "16px";
        }
    }

    function cargarPreferencia() {
        // document.cookie devuelve las cookies juntas, separadas por punto y coma.
        // split crea un array; trim quita los espacios que separan cada cookie.
        const cookies = document.cookie.split(";");
        for (let i = 0; i < cookies.length; i = i + 1) {
            if (cookies[i].trim() === "letraSumas=grande") {
                selectorLetra.value = "grande";
            }
        }
        aplicarLetra();
    }

    function guardarPreferencia() {
        aplicarLetra();
        // max-age son segundos: 30 días. path=/ permite leerla en todo este sitio.
        // SameSite=Lax limita el envío de esta cookie desde otros sitios.
        document.cookie = "letraSumas=" + selectorLetra.value + "; max-age=2592000; path=/; SameSite=Lax";
        const cookies = document.cookie.split(";");
        let guardada = false;
        for (let i = 0; i < cookies.length; i = i + 1) {
            if (cookies[i].trim() === "letraSumas=" + selectorLetra.value) {
                guardada = true;
            }
        }
        if (guardada) {
            mensajePreferencia.textContent = "Preferencia guardada durante 30 días.";
        } else {
            mensajePreferencia.textContent = "El navegador no ha guardado la preferencia.";
        }
    }
```

```
selectorLetra.addEventListener("change", guardarPreferencia);
cargarPreferencia();
```

estilos.css

```
/* Incluye el relleno y el borde dentro del ancho de cada elemento.
   Esto evita que los elementos ocupen más espacio del esperado. */
* {
    box-sizing: border-box;
}

/* Estilos generales de la página: letra, fondo y separación exterior. */
body {
    margin: 0;
    padding: 24px;
    font-family: Arial, sans-serif;
    background-color: #eef2f6;
    color: #1f2937;
}

/* Centra el panel y limita su ancho para facilitar la lectura. */
main {
    max-width: 600px;
    margin: 40px auto;
    padding: 24px;
    background-color: white;
    border-radius: 12px;
}

h1 {
    margin-top: 0;
}

p {
    line-height: 1.5;
}

/* El cursor indica que el botón se puede pulsar. */
button {
    padding: 12px 20px;
    border: none;
    border-radius: 6px;
    background-color: #1d4ed8;
    color: white;
    font: inherit;
    cursor: pointer;
}

/* Cambia el color al pasar el ratón por encima del botón. */
button:hover {
    background-color: #1e40af;
}

/* Destaca el botón cuando se selecciona usando el teclado. */
button:focus-visible {
    outline: 3px solid #111827;
    outline-offset: 3px;
}

/* Coloca cada etiqueta encima de su campo y separa los datos del formulario. */
label {
    display: block;
    margin-bottom: 6px;
}

input, select {
    display: block;
    width: 100%;
    padding: 10px;
    margin-bottom: 16px;
    border: 1px solid #6b7280;
    border-radius: 6px;
    font: inherit;
}

/* Separa el panel del mensaje y deja espacio alrededor de los datos. */
#panel-jugador {
    margin-top: 20px;
    padding: 16px;
    border: 1px solid #6b7280;
    border-radius: 6px;
    overflow-wrap: anywhere; /* Evita que un nombre largo se salga del panel. */
}
```

```css
/* Deja separación entre botones y permite leer un historial largo. */
#zona-juego {
    margin-top: 24px;
    overflow-wrap: anywhere;
}

#zona-juego button {
    margin-bottom: 8px;
}

/* Un botón gris indica que la suma ya se ha acertado. */
button:disabled {
    background-color: #6b7280;
    cursor: default;
}
```